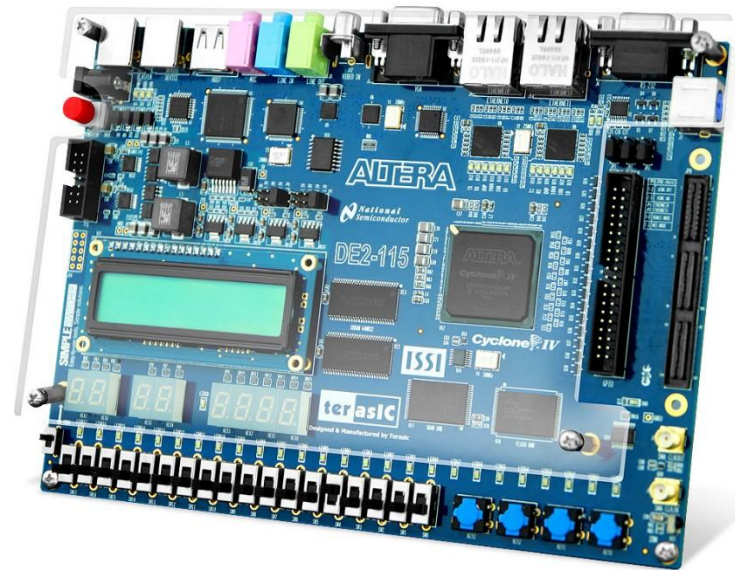


COEN-230

Digital System Design I

Dr. Fadi Alzhouri

Department of Electrical and Computer Engineering
Gust University





Outline

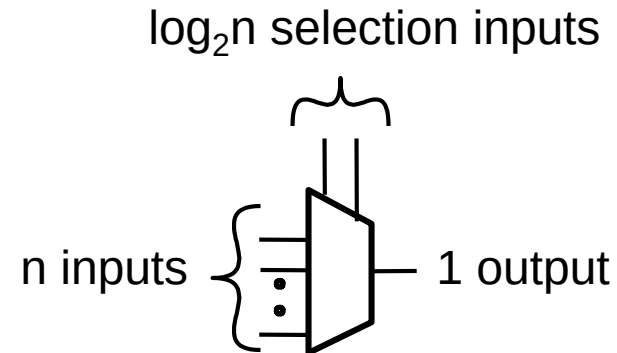
- Multiplexers
- Logic synthesis with multiplexers
- Shannon Expansion concept
- Decoders
- Encoders
- Code converters
- Arithmetic Comparator
- Practice



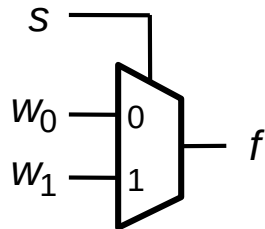
Multiplexers

• multiplexer

- 1 binary output
- $\log_2 n$ binary selection inputs
- n binary inputs (binary input = 1-bit input)
- Function: one of n inputs is placed onto output
- Called **n -to-1** multiplexer



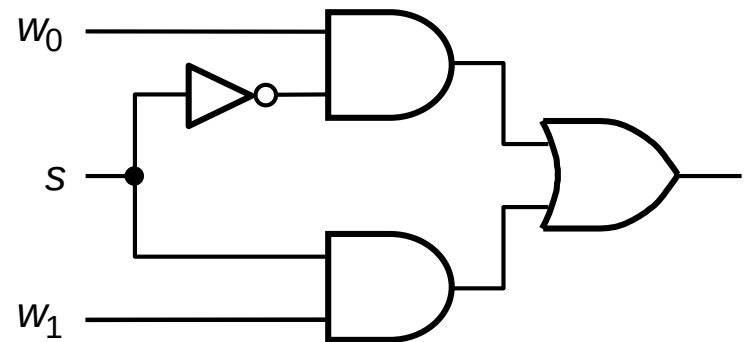
■ 2-to-1 multiplexer



(a) Graphical symbol

s	f
0	w_0
1	w_1

(b) Truth table



(c) Sum-of-products circuit



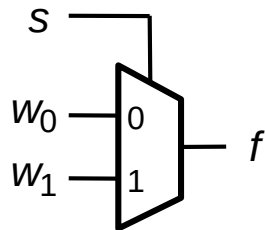
Multiplexers

Based on the value of the select signals, only one of the inputs will be propagated to the output

Show original truth table to implement 2-1 Mux

● Multiplexers have the following properties:

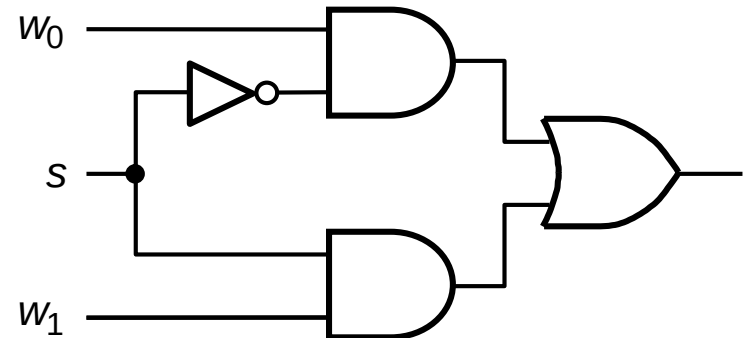
- $N = \#$ of inputs (usually a power of 2),
- $\#$ of select lines is $\log_2 N$



(a) Graphical symbol

s	f
0	w_0
1	w_1

(b) Truth table

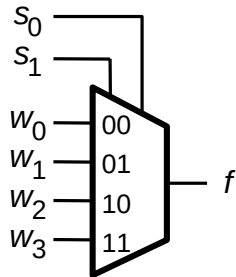


(c) Sum-of-products circuit



4-1 Multiplexer

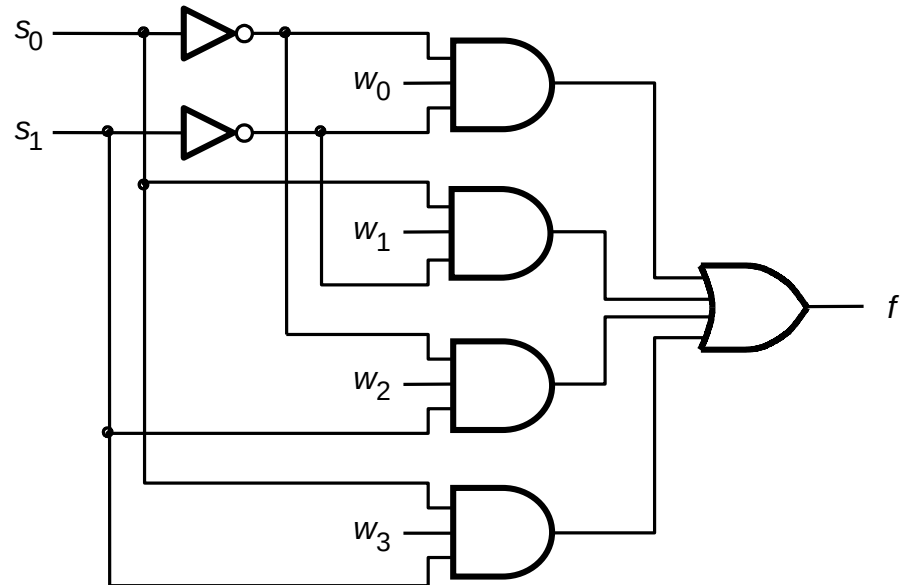
Larger multiplexers can be implemented in a similar fashion
Note how to write shorthand truth table



(a) Graphic symbol

s_1	s_0	f
0	0	w_0
0	1	w_1
1	0	w_2
1	1	w_3

(b) Truth table



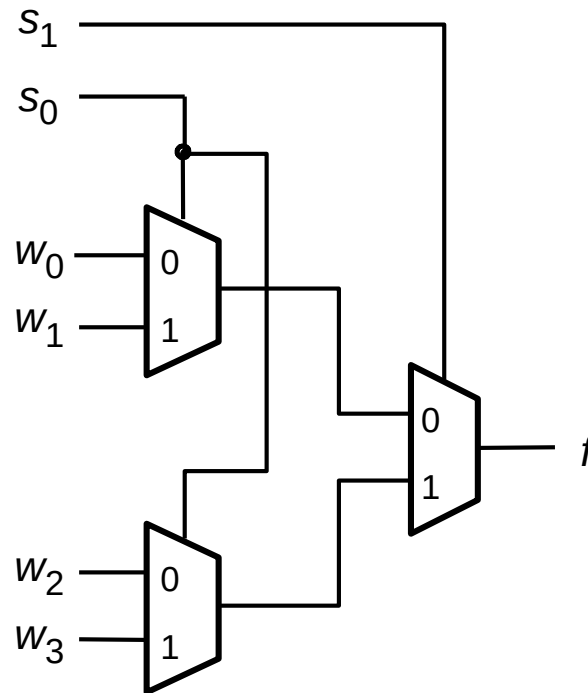
(c) Circuit



Alternative Approach

Smaller multiplexers can also be used to build larger ones

Example of 4-1 MUX. The MSbit selects one of the two MUXes and the remaining bit selects the inputs in each.



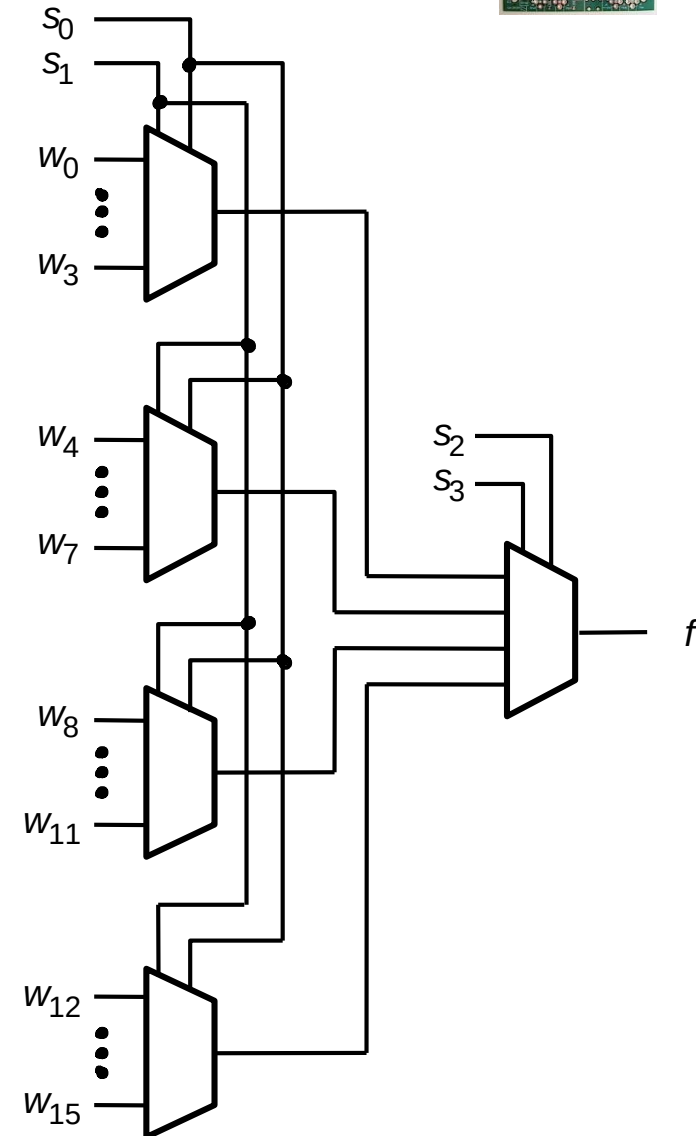
16-1 MUX

Built using 4-1 MUXes

Note how the selection bits are grouped in order to select among the 4-1 MUXes:

s_1s_0 are used for least significant 2 bits

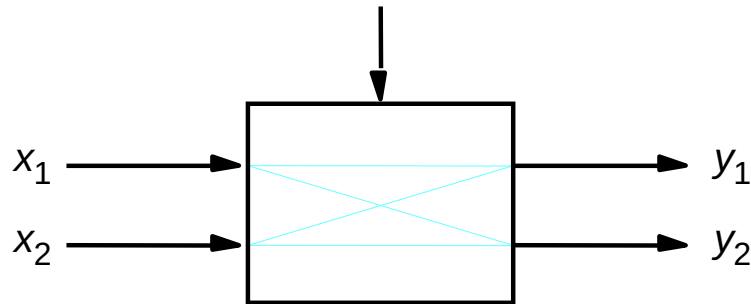
s_3s_2 are used for most significant bits



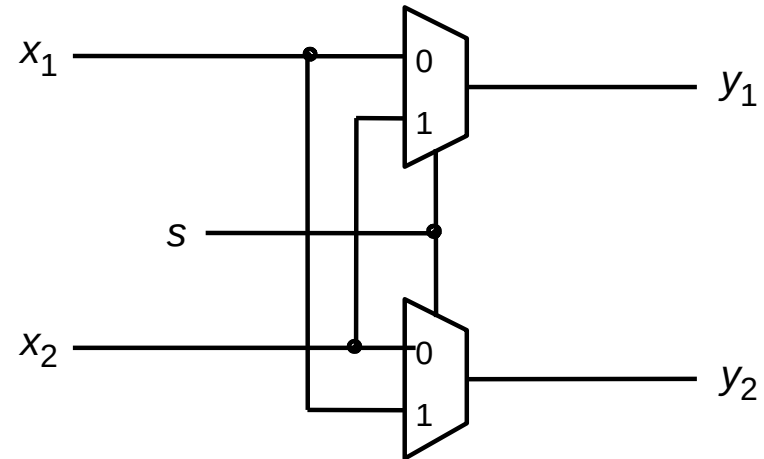


MUX Applications (1)

2x2 crossbar switch

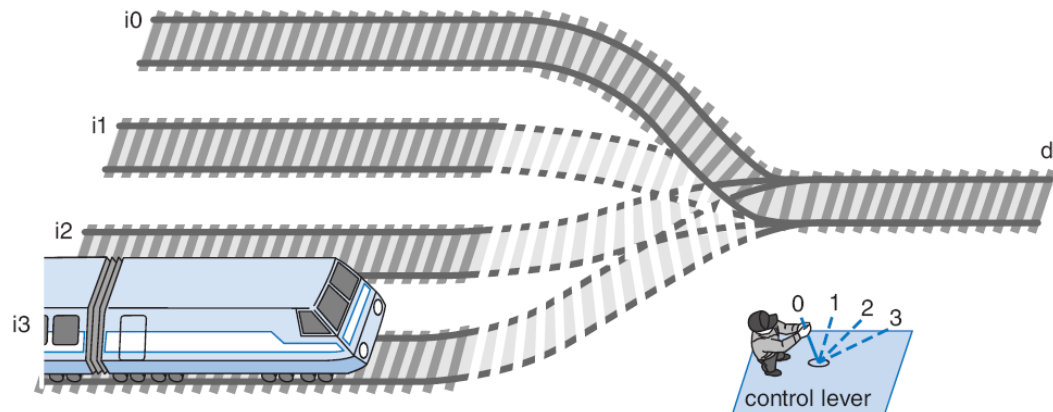


(a) A 2x2 crossbar switch



(b) Implementation using multiplexers

4-1 mux

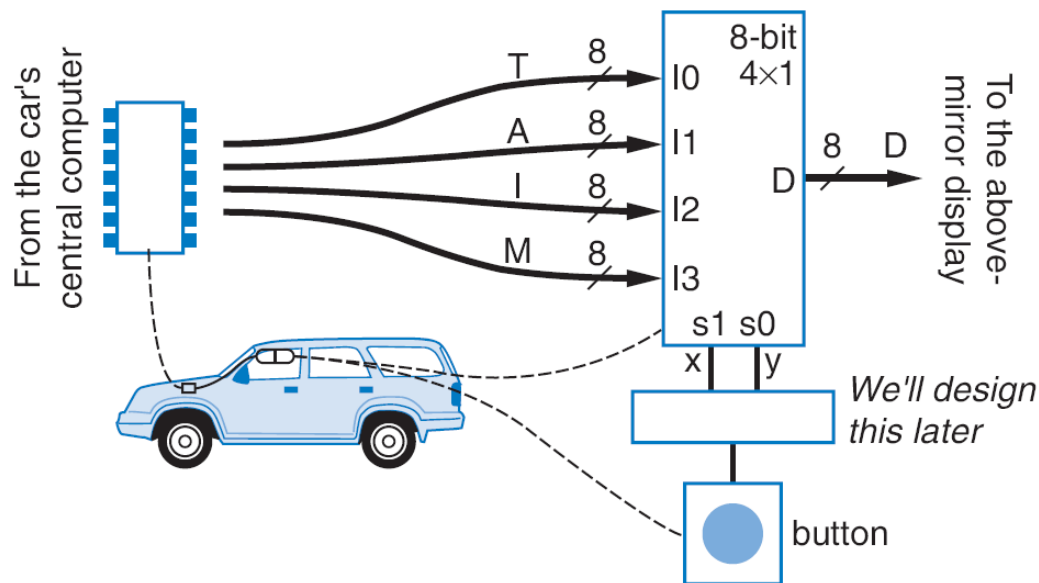




MUX Applications (2)

● Four possible display items

- Temperature (T), Average miles-per-gallon (A), Instantaneous mpg (I), and Miles remaining (M) -- each is 8-bits wide
- Choose which to display using two inputs x and y
- Use 8-bit 4x1 mux





Synthesis of Logic Circuits Using MUXes:

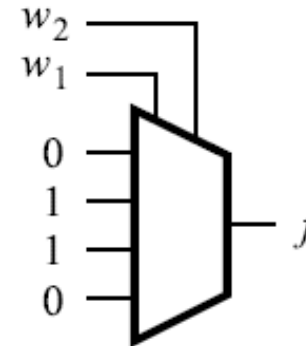
Example 1

Muxes can be used to implement **any** logic function:

Example: $f(w_1, w_2) = w_1'w_2 + w_1w_2'$ or $(w_1 \oplus w_2)$

w_1	w_2	f
0	0	0
0	1	1
1	0	1
1	1	0

4-1 MUX
→



The values of f in each row of the truth table are connected as constants to the multiplexer data inputs.

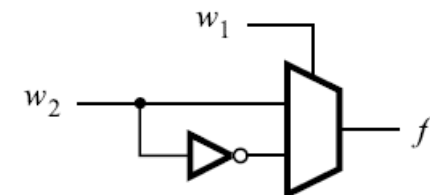
● Not very efficient. Can you use a single 2-1 MUX?

- Hint: Use w_1 as the selection switch.

w_1	w_2	f
0	0	0
0	1	1
1	0	1
1	1	0

→

w_1	f
0	w_2
1	$\bar{w}_2$





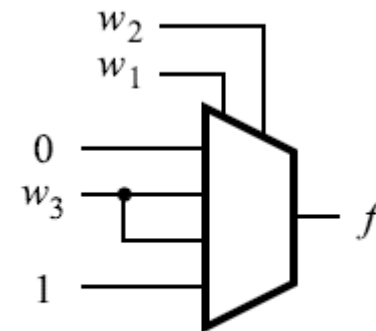
Synthesis of Logic Circuits:

Example 2

Implement the majority function using one 4-1 MUX (i.e. the output should be one if the number of 1's is at least 2).

w_1	w_2	w_3	f		w_1	w_2	f
0	0	0	0	}	0	0	0
0	0	1	0		0	1	w_3
0	1	0	0	}	1	0	w_3
0	1	1	1		1	1	1
1	0	0	0	}			
1	0	1	1				
1	1	0	1	}			
1	1	1	1				

(a) Modified truth table



(b) Circuit



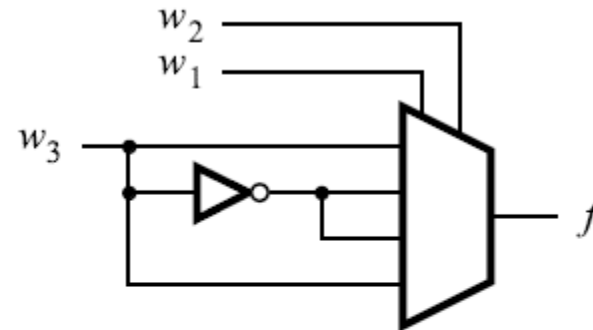
Synthesis of Logic Circuits:

Example 3

Implement a 3-input XOR using one 4-1 MUX

w_1	w_2	w_3	f	
0	0	0	0	} w_3
0	0	1	1	
0	1	0	1	} $\bar{w}_3$
0	1	1	0	
1	0	0	1	} $\bar{w}_3$
1	0	1	0	
1	1	0	0	} w_3
1	1	1	1	

(a) Truth table



(b) Circuit



Synthesis using Shannon's Expansion Theorem

The inputs to MUXes may not only be constants or variables, but also functions

Possible using Shannon's expansion theorem:

$$f(w_1, w_2, \dots, w_n) = w_1 \cdot f(0, w_2, \dots, w_n) + w_1' \cdot f(1, w_2, \dots, w_n)$$

$f(0, w_2, \dots, w_n)$ is labeled as $f_{w_1'}$

$f(1, w_2, \dots, w_n)$ is labeled as f_{w_1}

Thus, Shannon's expansion of any function can be written as

$$f(w_1, w_2, \dots, w_n) = w_x' \cdot f_{w_x'} + w_x \cdot f_{w_x}$$

Both f_{w_x} and $f_{w_x'}$ are called co-factor of w_1 and w_1' respectively



How to Use Shannon's theorem

To represent any function in this form, do the following:

Choose the variable to decomposed the function with respect to
(i.e. w_x)

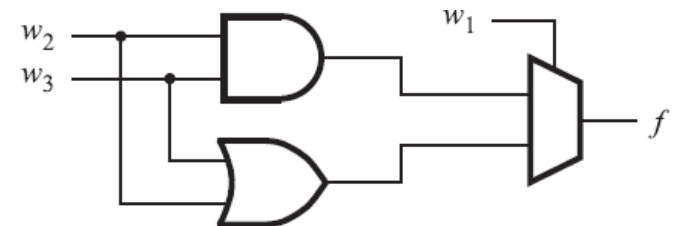
Substitute value "0" for w_x in the function to find $f_{w_x'}$

Substitute value "1" for w_x in the function to find f_{w_x}

Write the function in Shannon expansion form

Example: Majority Function

$$\begin{aligned} f(w_1, w_2, w_3) &= w_1 w_2 + w_1 w_3 + w_2 w_3 \\ &= \bar{w}_1 f_{\bar{w}_1} + w_1 f_{w_1} \\ &= \bar{w}_1 () + w_1 () \\ &= \bar{w}_1 (w_2 w_3) + w_1 (w_2 + w_3) \end{aligned}$$



(b) Circuit



Synthesis using Shannon's Expansion Theorem

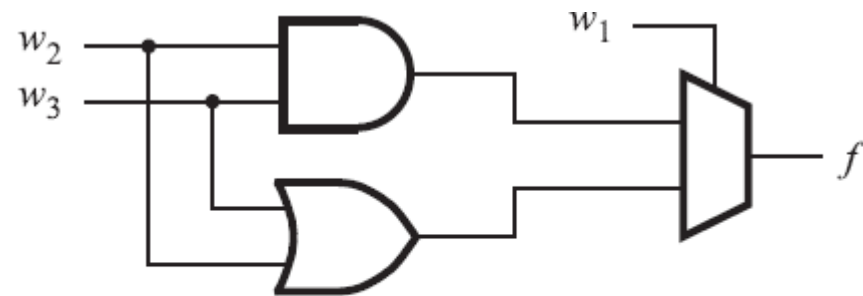
The previous example is designed here

- Majority Function = $\bar{w}_1(w_2w_3) + w_1(w_2 + w_3)$
Implementation using one 2-1 MUX

w_1	w_2	w_3	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

w_1	f
0	w_2w_3
1	$w_2 + w_3$

(a) Truth table



(b) Circuit



More examples

Implement

$$f = \bar{w}_1\bar{w}_3 + w_1w_2 + w_1w_3$$

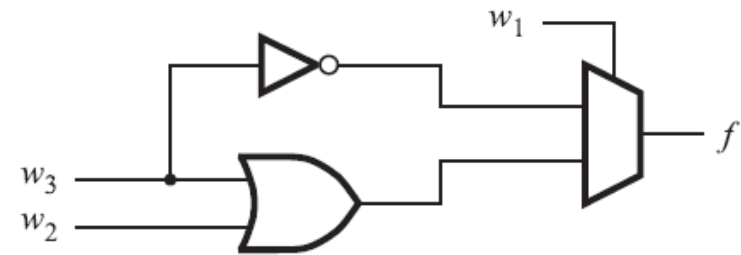
using 2-1 and 4-1 MUXes.

Expansion based on w_1 gives

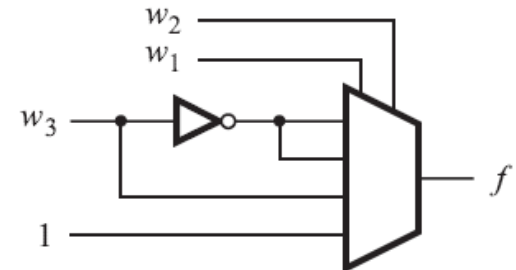
$$\begin{aligned} f &= \bar{w}_1f_{\bar{w}_1} + w_1f_{w_1} \\ &= \bar{w}_1(\bar{w}_3) + w_1(w_2 + w_3) \end{aligned}$$

- Expansion based on w_1 and w_2 gives

$$\begin{aligned} f &= \bar{w}_1\bar{w}_2f_{\bar{w}_1\bar{w}_2} + \bar{w}_1w_2f_{\bar{w}_1w_2} + w_1\bar{w}_2f_{w_1\bar{w}_2} + w_1w_2f_{w_1w_2} \\ &= \bar{w}_1\bar{w}_2(\bar{w}_3) + \bar{w}_1w_2(\bar{w}_3) + w_1\bar{w}_2(w_3) + w_1w_2(1) \end{aligned}$$



(a) Using a 2-to-1 multiplexer



(b) Using a 4-to-1 multiplexer



Shannon Expansion Application

Depending on the variable(s) we choose for decomposition, the cost of resulting implementation might be different

Find the decomposition of the function

for each of the variables w_1 , w_2 and w_3 . $f = \overline{w}_1 w_3 + w_2 \overline{w}_3$.

- The function can be decomposed into:

$$\overline{w}_1(w_3 + w_2) + w_1(w_2 \overline{w}_3)$$

$$\overline{w}_2(\overline{w}_1 w_3) + w_2(\overline{w}_1 + \overline{w}_3)$$

$$\overline{w}_3(w_2) + w_3(\overline{w}_1)$$

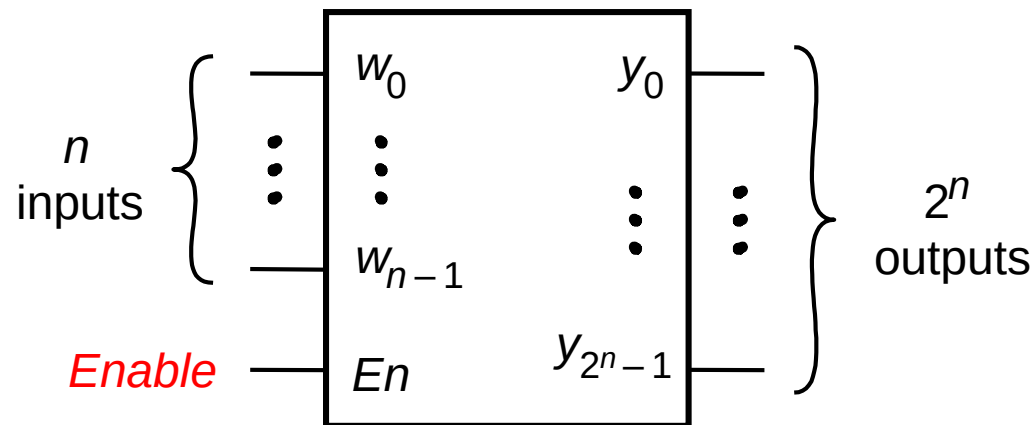
- Obviously, the last implementation is the most cost-effective one.



Decoders

● Decoder

- n binary inputs
- 2^n binary outputs
- Function: decode encoded information
 - If enable=1, one output is asserted high, the other outputs are asserted low
 - If enable=0, all outputs asserted low
- Called **n -to- 2^n** decoder
- Decoders are often used for RAM/ROM addressing, instruction decoding, timing generation and logic implementations



. An n -to- 2^n binary decoder.



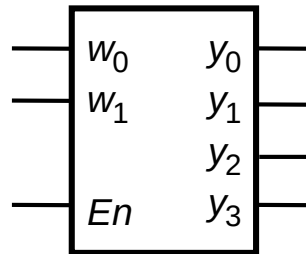
2-to-4 Decoder

Example: A 2-to-4 Decoder

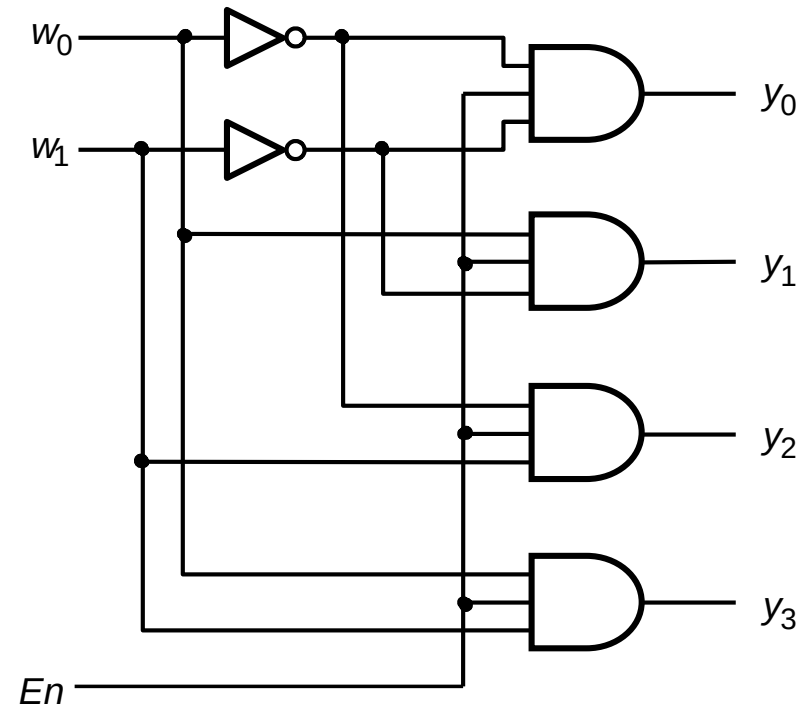
- Inputs: w_0 & w_1 . They represent a two-bit number that causes the decoder to assert one of the outputs.
- Outputs: $y_0, \dots, y_3$

En	w_1	w_0	y_0	y_1	y_2	y_3
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1
0	x	x	0	0	0	0

(a) Truth table



(b) Graphical symbol



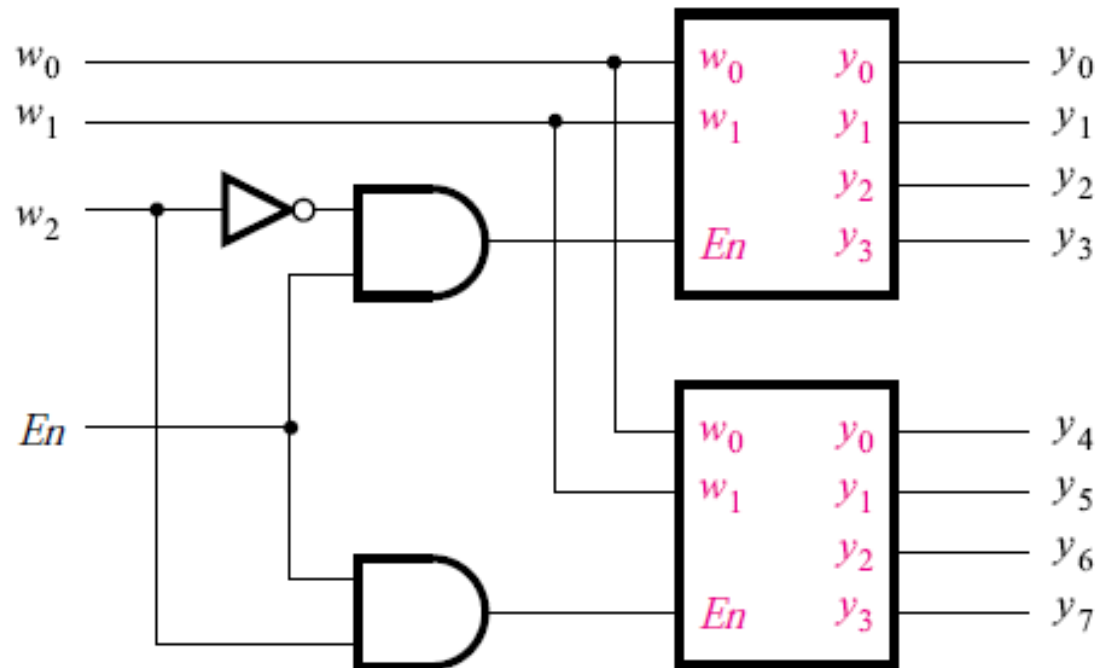
(c) Logic circuit



Synthesis of Large Decoders Using Smaller Ones

Larger decoders can be created from smaller ones

See how one input is used as an enable signal for 2-to-4 decoder



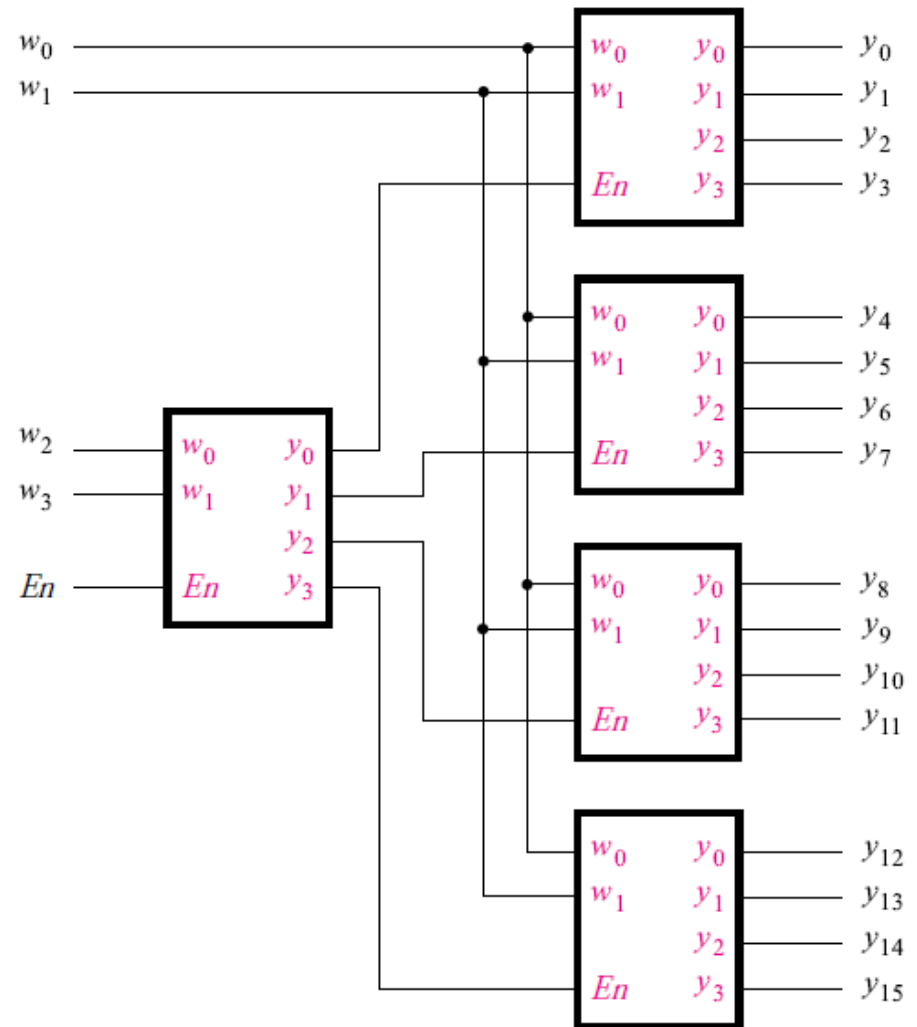


4-to-16 Decoder

Use decoder tree to construct this unit

Note:

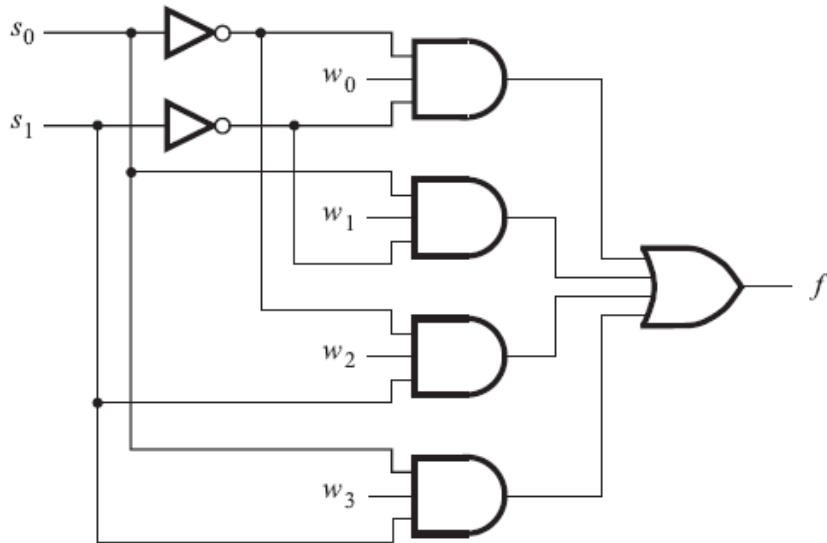
How most significant bits are use to enable various 2-4 decoders



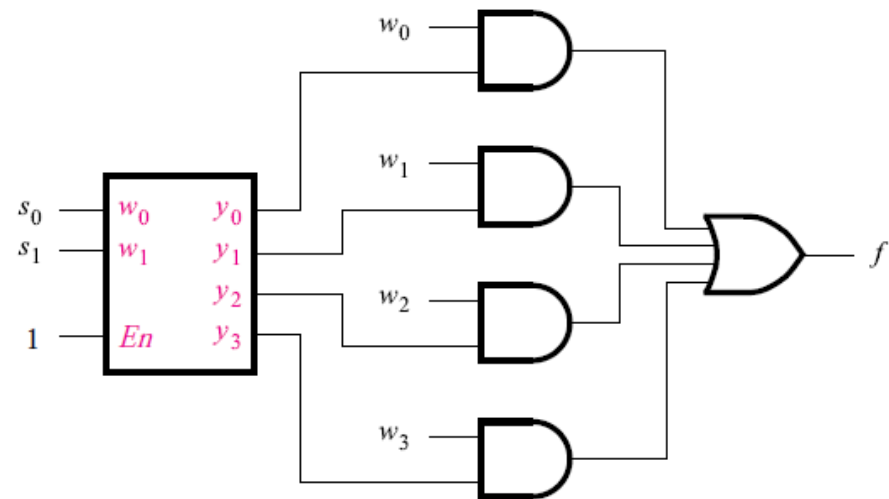


Using Decoders to Implement MUXes

How can we construct a MUX using decoders?



Sum-of-products implementation of the 4-to-1 multiplexer

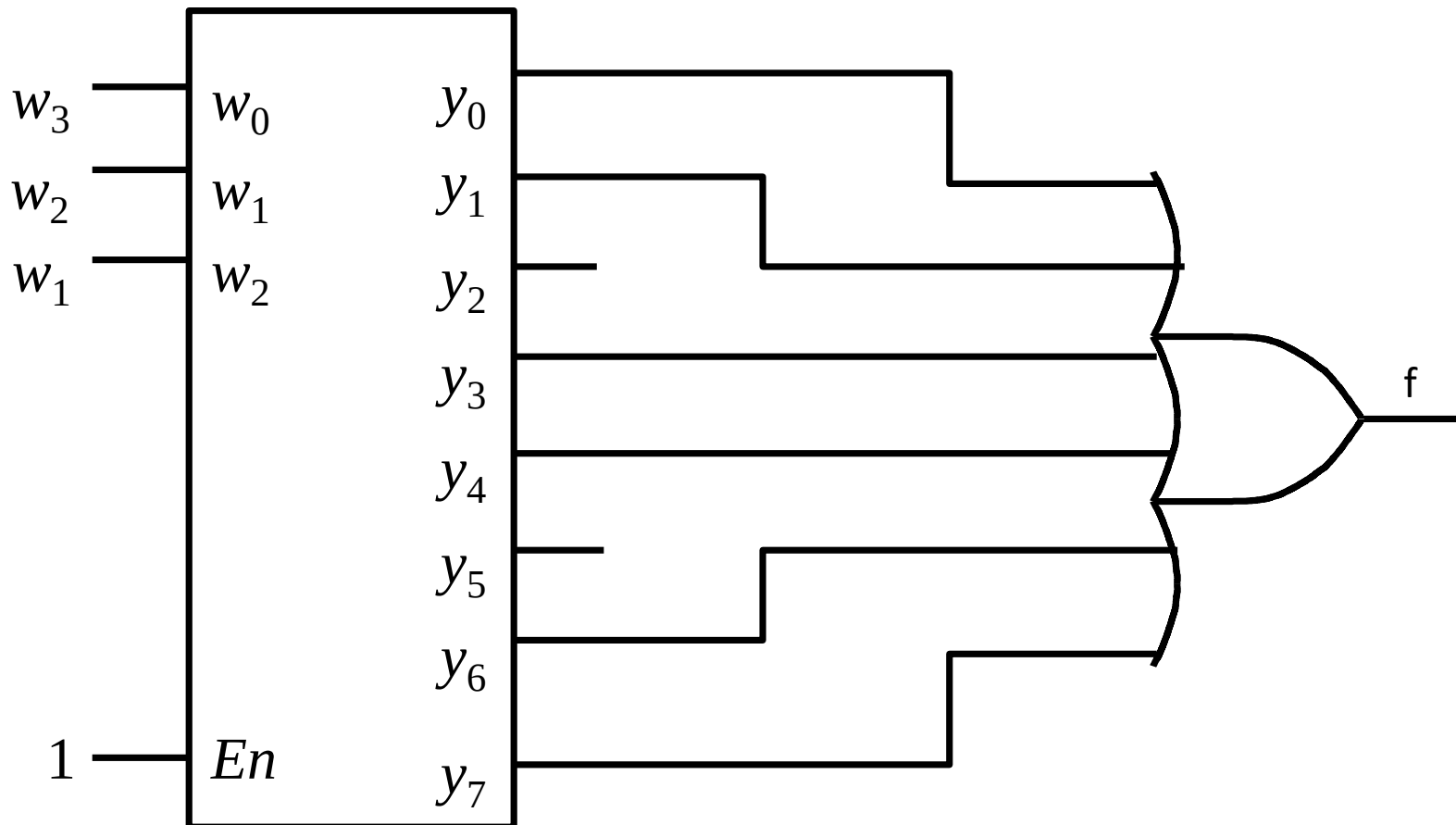


Implementation using a decoder



Using Decoders to Implement logic functions

Implement function $f(w_1, w_2, w_3) = \sum m(0, 1, 3, 4, 6, 7)$ by using 3-to-8 decoder and an OR gate



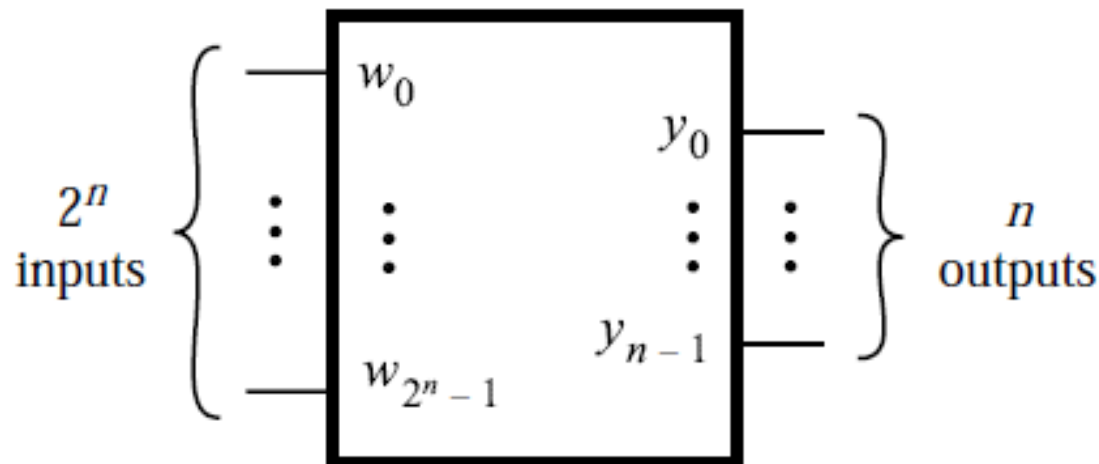


Encoders

Performs opposite function of a decoder

Encodes information from 2^n inputs into an n -bit code

The output contains the binary number representing the input equal to 1



Encoders are used to reduce the number of bits needed to represent given information.

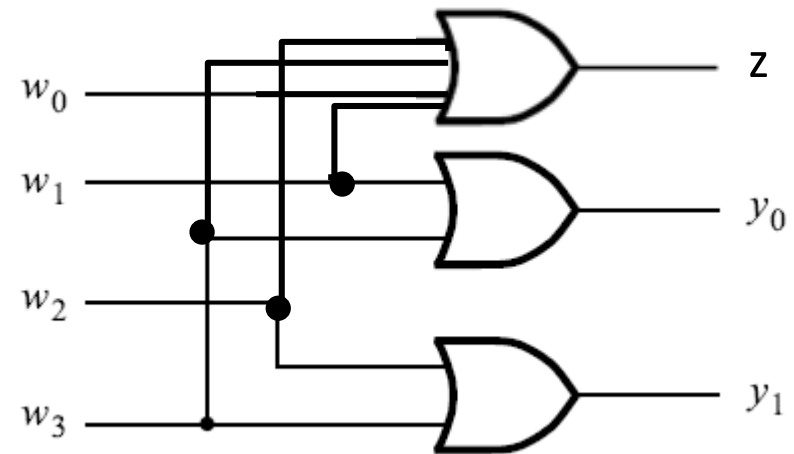
I.e. during transmission or storage



4-2 Encoder

Example: A 4-to-2 binary encoder

w_3	w_2	w_1	w_0	y_1	y_0	z
0	0	0	1	0	0	1
0	0	1	0	0	1	1
0	1	0	0	1	0	1
1	0	0	0	1	1	1



All input patterns that have multiple inputs set to 1 are not shown in the truth table, and they are treated as don't-care conditions.

What if more than one input is equal to 1?



Priority Encoders

The output contains the binary number representing the **highest priority** input that is equal to 1

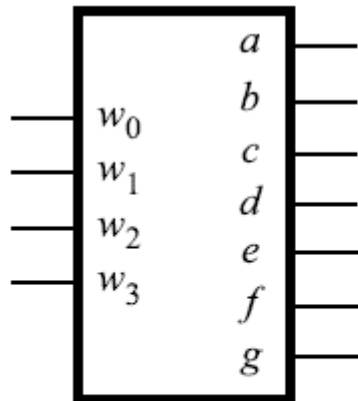
- Note that w_3 here represents the highest priority followed by w_2 ...etc.
- When an input with a high priority is asserted, the other inputs with lower priority are ignored.
- Signal Z represent the case when none is 1.

w_3	w_2	w_1	w_0	y_1	y_0	z
0	0	0	0	d	d	0
0	0	0	1	0	0	1
0	0	1	x	0	1	1
0	1	x	x	1	0	1
1	x	x	x	1	1	1

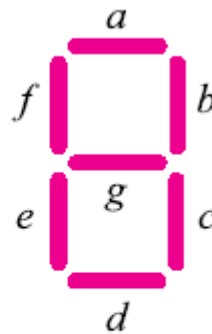


Code Converters

- Decoders & Encoders convert from one code type to another (binary to one-hot encoding, or the opposite, 7-segment code conversion)



(a) Code converter



(b) 7-segment display

w_3	w_2	w_1	w_0	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	1	1	1	0	1	1	1
1	0	1	1	0	0	1	1	1	1	1
1	1	0	0	1	0	0	1	1	1	0
1	1	0	1	0	1	1	1	1	0	1
1	1	1	0	1	0	0	1	1	1	1
1	1	1	1	1	0	0	0	1	1	1

(c) Truth table



Arithmetic Comparison Circuits

We've seen arithmetic circuits that perform addition, subtraction, and multiplication of binary numbers. Another useful type of arithmetic circuit **compares** the relative sizes of two binary numbers.

Such a circuit is called a **comparator**.

The comparator has two n -bit inputs, A and B , which represent **unsigned** binary numbers. It produces three outputs, called $AeqB$ ($A=B$), $AgtB$ ($A>B$), and $AltB$ ($A<B$).

The comparator can be designed by creating a truth table that specifies the three outputs as functions of A and B . However, even for moderate values of n , the truth table is large.



Building a comparator for n=4 bit

Let $A = a_3a_2a_1a_0$ and $B = b_3b_2b_1b_0$. When are these numbers equal?

Answer: When $a_3 = b_3$ and $a_2 = b_2$ and $a_1 = b_1$ and $a_0 = b_0$

So, define a set of intermediate signals called i_3, i_2, i_1 , and i_0 . Each signal, i_k , is 1 if the bits of A and B with the same index are equal.

Thus, $i_k = a_k \oplus b_k$.

The comparator's $AeqB$ output is then given by

$$AeqB = i_3 \cdot i_2 \cdot i_1 \cdot i_0$$



Building a comparator for n=4 bit (cont.)

Let $A = a_3a_2a_1a_0$ and $B = b_3b_2b_1b_0$. When is A bigger than B?

Answer: An expression for the $AgtB$ output can be derived by considering the bits of A and B in the order from the most-significant bit to the least-significant bit.

If $a_k = 0$ and $b_k = 1$, then $A < B$. But if $a_k = 1$ and $b_k = 0$, then $A > B$. And so on...

The $AgtB$ output is defined by

$$AgtB = a_3\bar{b}_3 + i_3a_2\bar{b}_2 + i_3i_2a_1\bar{b}_1 + i_3i_2i_1a_0\bar{b}_0$$

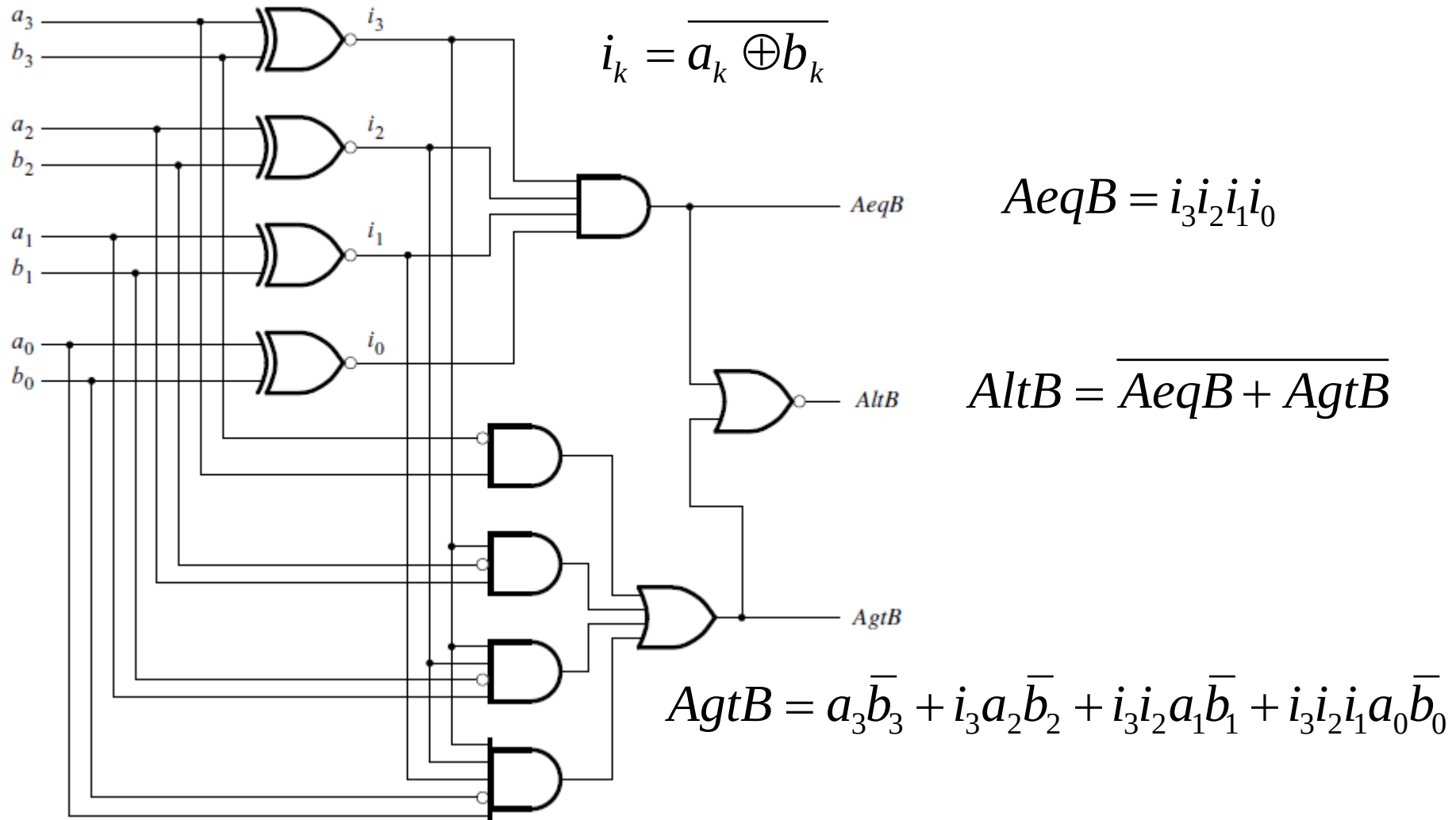
Let $A = a_3a_2a_1a_0$ and $B = b_3b_2b_1b_0$. When is A smaller than B?

Answer: When it's neither equal nor bigger. Thus,

$$AltB = \overline{AeqB + AgtB}$$



Arithmetic Comparison Circuit





Practice

Implement the function

$$f(w_1, w_2, w_3, w_4, w_5) = \overline{w_1}\overline{w_2}\overline{w_4}\overline{w_5} + w_1w_2 + w_1w_3 + w_1w_4 + w_3w_4w_5$$

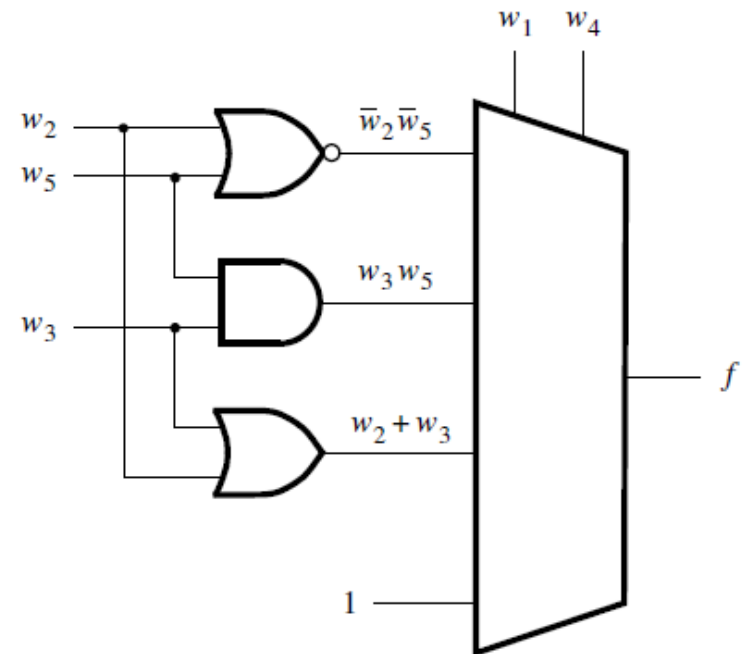
by using a 4-to-1 multiplexer and as few other gates as possible.

How many selection signals?

Which ones?

● Answer

- Since variables w_1 and w_4 appear in **more product terms** in the expression for f than the other three variables, perform Shannon's expansion with respect to these two variables.





Practice 2

We showed that any logic function can be decomposed using Shannon's expansion theorem.

For a four-variable function, $f(w_1, \dots, w_4)$, the expansion with respect to w_1 is

$$f(w_1, \dots, w_4) = \bar{w}_1 f_{\bar{w}_1} + w_1 f_{w_1}$$

- If the decomposition yields $f_{w_1} = 0$, then the multiplexer in the figure can be replaced by a single logic gate. **How?**
- Repeat this for the case where $f_{w_1} = 1$.

